1. Definición del Layout de Parámetros (APVTS)

Se ha implementado la arquitectura base de los controles del plugin, utilizando el sistema de gestión de estado de JUCE para centralizar la comunicación entre el procesador y la interfaz de usuario.

EQ_SharedCode → EQAudioProcessor

```cpp
juce::AudioProcessorValueTreeState::ParameterLayout EQAudioProcessor::createParameterLayout()
{
    juce::AudioProcessorValueTreeState::ParameterLayout layout;

    layout.add(std::make_unique<juce::AudioParameterFloat>("LowCut Frequency",
                                                           "Low Cut Frequency",
                                                           juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
                                                           20.0f));

    layout.add(std::make_unique<juce::AudioParameterFloat>("HighCut Frequency",
                                                           "HighCut Frequency",
                                                           juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
                                                           20000.0f));

    layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Frequency",
                                                           "Peak Frequency",
                                                           juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
                                                           750.0f));

    layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Gain",
                                                           "Peak Gain",
                                                           juce::NormalisableRange<float>(-24.0f, 24.0f, 0.5f, 1.0f),
                                                           0.0f));

    layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Quality",
                                                           "Peak Quality",
                                                           juce::NormalisableRange<float>(0.1f, 10.0f, 0.05f, 1.0f),
                                                           1.0f));
    juce::StringArray stringArray;
    for (int i = 0; i < 4; ++i)
    {
        juce::String str;
        str << (12 + i * 12);
        str << " dB/Oct";
        stringArray.add(str);
    }

    layout.add(std::make_unique<juce::AudioParameterChoice>("LowCut Slope", "LowCut Slope", stringArray,0));
    layout.add(std::make_unique<juce::AudioParameterChoice>("HighCut Slope", "HighCut Slope", stringArray, 0));

    return layout;
}
```

Parámetros y Lógica:

- Bandas de Frecuencia: Se definen tres puntos de control (LowCut, HighCut y Peak) con un rango de 20 Hz a 20 kHz. Se utiliza una resolución de 1.0 Hz.

- Control de Ganancia (Peak): Rango de ±24 dB con pasos de 0.5 dB, permitiendo un control detallado sobre la amplitud de la banda paramétrica.

- Factor de Calidad (Q): Rango de 0.1 a 10.0 con pasos de 0.05, definiendo el ancho de banda del filtro de campana.

- Pendientes de Corte (Slopes): Implementación de una lógica de selección discreta mediante un bucle for que genera opciones de 12, 24, 36 y 48 dB/Oct para los filtros de paso alto y paso bajo.

Funciones y Variables Usadas:

- *createParameterLayout()*: Método encargado de instanciar y retornar el objeto ParameterLayout necesario para la inicialización del AudioProcessorValueTreeState.

- *juce::AudioParameterFloat*: Clase para parámetros de valor continuo.
    - parameterID: String identificador único para automatización.
    - parameterName: Nombre legible por el usuario en el DAW.
    - NormalisableRange<float>: Objeto que define el comportamiento del slider (mín, máx, intervalo, skew).
    - defaultValue: Valor inicial asignado al cargar el plugin.

- *juce::AudioParameterChoice*: Clase para parámetros de selección múltiple (menús desplegables).
    - choices: StringArray que contiene las etiquetas de texto de las opciones disponibles.
    - defaultIndex: Índice de la opción seleccionada por defecto.

- *juce::NormalisableRange<float>*:
    - rangeStart / rangeEnd: Límites del parámetro.
    - intervalValue: Incremento mínimo del valor.
    - skewFactor: Factor de asimetría (1.0 para respuesta lineal)

La estructura del APVTS permite una gestión de parámetros thread-safe, esencial para la sincronización entre el hilo de audio y el de la interfaz (GUI). El uso de NormalisableRange con pasos específicos garantiza que la cuantización de los valores de control sea coherente con los requisitos de precisión de un procesador de señal profesional.

2. Configuración de la Cadena de Procesado (DSP Chain)

Se ha definido la infraestructura de procesamiento de señal utilizando el módulo DSP de JUCE, estructurando los filtros en cadenas de procesado mono para gestionar el audio en estéreo de forma independiente.

EQ_SharedCode | EQAudioProcessor

```
58
59      private:
60          using Filter = juce::dsp::IIR::Filter<float>;
61
62          using CutFilter = juce::dsp::ProcessorChain<Filter, Filter, Filter, Filter>;
63
64          using MonoChain = juce::dsp::ProcessorChain<CutFilter, Filter, CutFilter>;
65
66          MonoChain leftChain, rightChain;
```

Parámetros y Lógica:

- Jerarquía de Filtros: Se utiliza un alias Filter para filtros IIR de precisión simple.

- Estructura de Corte (CutFilter): Se define un ProcessorChain compuesto por 4 filtros en serie. Esta arquitectura permite sumar pendientes de 12 dB/Oct por cada filtro, logrando los cortes de hasta 48 dB/Oct definidos en el APVTS.

- Cadena Mono (MonoChain): Estructura final que organiza el flujo de señal en tres etapas: LowCut (CutFilter), Peak (Filter) y HighCut (CutFilter).

- Instanciación Estéreo: Se declaran dos instancias de MonoChain (leftChain y rightChain) para procesar los canales izquierdo y derecho por separado.

Funciones y Variables Usadas:

- *juce::dsp::IIR::Filter<float>*: Clase base para filtros de respuesta al impulso infinita.
- *juce::dsp::ProcessorChain*: Plantilla de clase que permite encadenar múltiples procesadores DSP, gestionando automáticamente el paso de muestras entre ellos.
- *using (Type Aliasing)*: Utilizado para simplificar la declaración de tipos complejos de plantillas (templates), mejorando la legibilidad del código.

La implementación establece una topología de filtros en serie. La modularidad de ProcessorChain permite tratar cada banda como una unidad lógica, facilitando la expansión de la pendiente del filtro de corte mediante la concatenación de instancias IIR de segundo orden.

3. Inicialización en prepareToPlay

Se ha implementado la preparación del motor DSP, asegurando que todos los componentes de la cadena de audio conozcan las especificaciones del entorno antes de comenzar la reproducción.

EQ_SharedCode (Ámbito global)

```
//==============================================================================
void EQAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
{
    // Use this method as the place to do any pre-playback
    // initialisation that you need..

    juce::dsp::ProcessSpec spec;

    spec.maximumBlockSize = samplesPerBlock;

    spec.numChannels = 1;

    spec.sampleRate = sampleRate;

    leftChain.prepare(spec);
    rightChain.prepare(spec);
}
```

Parámetros y Lógica:

- Configuración del DSP: Se utiliza un objeto ProcessSpec para pasar la frecuencia de muestreo y el tamaño máximo de bloque a las cadenas de filtrado.

- Sincronización de Canales: Cada MonoChain se inicializa con un solo canal (numChannels = 1), ya que el procesado estéreo se divide en dos cadenas mono independientes.

Funciones y Variables Usadas:

- *prepareToPlay()*: Método del AudioProcessor que el DAW llama antes de empezar el procesado de audio.

- *juce::dsp::ProcessSpec*: Estructura que contiene:
    - sampleRate: Frecuencia de muestreo actual (p. ej. 44100 Hz).
    - maximumBlockSize: Tamaño máximo de los bloques de audio que procesará el plugin.
    - numChannels: Número de canales que manejará ese procesador específico.

- *prepare()*: Método que reserva memoria y precalcula coeficientes necesarios en cada elemento de la cadena DSP.

Antes de que empiece a sonar la música, el programa necesita saber a qué velocidad va el audio (sample rate) y cuánta información le llegará de golpe. Esta función le da esas instrucciones al ecualizador para que esté listo y no haya errores o ruidos extraños al dar al "play".

4. Implementación del Bucle de Procesado (processBlock)

Se ha configurado el núcleo del procesamiento en tiempo real, donde el audio entrante se descompone en bloques mono para ser filtrado por las cadenas DSP correspondientes.

EQ_SharedCode → EQAudioProcessor → hasEditor() const

```cpp
void EQAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels  = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    // In case we have more outputs than inputs, this code clears any output
    // channels that didn't contain input data, (because these aren't
    // guaranteed to be empty - they may contain garbage).
    // This is here to avoid people getting screaming feedback
    // when they first compile a plugin, but obviously you don't need to keep
    // this code if your algorithm always overwrites all the output channels.
    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear (i, 0, buffer.getNumSamples());

    juce::dsp::AudioBlock<float> block(buffer);

    auto leftBlock = block.getSingleChannelBlock(0);
    auto rightBLock = block.getSingleChannelBlock(1);

    juce::dsp::ProcessContextReplacing<float> leftContext(leftBlock);
    juce::dsp::ProcessContextReplacing<float> rightContext(rightBLock);

    leftChain.process(leftContext);
    rightChain.process(rightContext);
}
```

Parámetros y Lógica:

- Gestión de Denormales: Se utiliza ScopedNoDenormals para evitar caídas de rendimiento en la CPU causadas por números denormales (underflow) en los cálculos recursivos de los filtros IIR.

- Abstracción de Bloques: Se convierte el AudioBuffer nativo en un AudioBlock, facilitando la compatibilidad con el módulo DSP de JUCE.

- Procesado Contextual: Se crean contextos de proceso para cada canal, permitiendo que los filtros modifiquen el audio directamente "in-place".

Funciones y Variables Usadas:

- *juce::ScopedNoDenormals*: Clase que optimiza el uso de la CPU durante el procesado evitando el manejo de números extremadamente cercanos a cero.

- *juce::dsp::AudioBlock<float>*: Clase que envuelve los datos del buffer de audio.
    - getSingleChannelBlock(index): Extrae un puntero y metadatos de un canal específico del bloque.

- *juce::dsp::ProcessContextReplacing<float>*: Contexto que gestiona la entrada y salida de datos sobre la misma dirección de memoria, optimizando el acceso a caché.

El bloque de proceso segmenta el flujo de datos estéreo en flujos mono independientes. El uso de contextos de reemplazo (Replacing) asegura una eficiencia máxima al procesar los coeficientes del filtro directamente sobre el buffer de entrada, minimizando la latencia y el overhead de memoria.

5. Estructura de Datos y Extracción de Parámetros

Se ha definido una estructura auxiliar y una función de utilidad para centralizar la obtención de valores desde el AudioProcessorValueTreeState, facilitando su uso en el bucle de audio.

```
EQ_SharedCode                                   EQAudioProcessor

struct ChainSettings
    {
    float peakFreq{ 0 }, peakGainInDecibels{ 0 }, peakQuality{ 1.0f };
    float lowCutFreq{ 0 }, highCutFreq{ 0 };
    int lowCutSlope{ 0 }, highCutSlope{ 0 };
};

ChainSettings getChainSettings(juce::AudioProcessorValueTreeState& apvts);
```

```
EQ_SharedCode        (Ámbito global)        getChainSettings(juce::AudioProcesso

ChainSettings getChainSettings(juce::AudioProcessorValueTreeState& apvts)
{
    ChainSettings settings;

    settings.peakFreq = apvts.getRawParameterValue("Peak Frequency")->load();
    settings.peakGainInDecibels = apvts.getRawParameterValue("Peak Gain")->load();
    settings.peakQuality = apvts.getRawParameterValue("Peak Quality")->load();
    settings.lowCutFreq = apvts.getRawParameterValue("LowCut Frequency")->load();
    settings.highCutFreq = apvts.getRawParameterValue("HighCut Frequency")->load();
    settings.lowCutSlope = apvts.getRawParameterValue("LowCut Slope")->load();
    settings.highCutSlope = apvts.getRawParameterValue("HighCut Slope")->load();

    return settings;
}
```

Parámetros y Lógica:

- Estructura ChainSettings: Agrupa todas las variables de control (frecuencias, ganancias, calidades y pendientes) en un solo objeto para evitar llamadas repetitivas y dispersas al APVTS.

- Función getChainSettings: Realiza la lectura de los valores actuales de los parámetros. Utiliza load() para garantizar una lectura atómica y segura desde el hilo de audio.

- Mapeo de Tipos: Las frecuencias y ganancias se extraen como float, mientras que los índices de las pendientes (slopes) se recuperan como int.

Funciones y Variables Usadas:

- *apvts.getRawParameterValue()*: Obtiene un puntero al valor en tiempo real de un parámetro mediante su ID.

- *load()*: Método de std::atomic que asegura que el valor leído sea el más reciente y consistente entre diferentes hilos de ejecución.

La implementación de una función de extracción dedicada desacopla la lógica de obtención de datos de la lógica de procesamiento. Esto mejora la mantenibilidad y asegura que el hilo de audio trabaje con una "instantánea" coherente de todos los controles al inicio de cada bloque.

6. Actualización de Coeficientes en Tiempo Real

Se ha integrado la lógica necesaria para transformar los parámetros de control en coeficientes matemáticos aplicables al filtro Peak, tanto en la inicialización como en el proceso continuo.

EQ_SharedCode → EQAudioProcessor → prepareToPlay(double sampleRate, int

```cpp
void EQAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
{
    // Use this method as the place to do any pre-playback
    // initialisation that you need..

    juce::dsp::ProcessSpec spec;

    spec.maximumBlockSize = samplesPerBlock;

    spec.numChannels = 1;

    spec.sampleRate = sampleRate;

    leftChain.prepare(spec);
    rightChain.prepare(spec);

    auto chainSettings = getChainSettings(apvts);
    auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(sampleRate, chainSettings.peakFreq,
        chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));

    *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
    *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
}
```

```cpp
void EQAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels  = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    // In case we have more outputs than inputs, this code clears any output
    // channels that didn't contain input data, (because these aren't
    // guaranteed to be empty - they may contain garbage).
    // This is here to avoid people getting screaming feedback
    // when they first compile a plugin, but obviously you don't need to keep
    // this code if your algorithm always overwrites all the output channels.
    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear (i, 0, buffer.getNumSamples());

    auto chainSettings = getChainSettings(apvts);
    auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(getSampleRate(), chainSettings.peakFreq,
        chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));

    *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
    *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
```

Parámetros y Lógica:

- Cálculo de Coeficientes: Se utiliza la función estática makePeakFilter para generar los coeficientes de un filtro de campana basados en la frecuencia central, el factor Q y la ganancia.

- Conversión de Unidades: Se transforma la ganancia de decibelios (dB) a ganancia lineal mediante decibelsToGain, requisito necesario para el motor DSP.

- Asignación a la Cadena: Se accede al elemento específico de la cadena de procesamiento (ChainPositions::Peak) y se actualizan sus coeficientes de forma sincronizada para los canales izquierdo y derecho.

Funciones y Variables Usadas:

- *juce::dsp::IIR::Coefficients<float>::makePeakFilter*: Calcula los coeficientes del filtro IIR basándose en:
    - sampleRate: Frecuencia de muestreo.
    - frequency: Frecuencia central de la campana.
    - Q: Factor de calidad (ancho de banda).
    - gain: Ganancia lineal.

- *leftChain.get<Index>()*: Función de plantilla que permite acceder a un procesador específico dentro de una ProcessorChain en tiempo de compilación.

- **coefficients = *newCoefficients*: Operación de desreferenciación para copiar los valores de los nuevos coeficientes en el puntero gestionado por el filtro.

La actualización de coeficientes en el processBlock permite que el filtro responda instantáneamente a los cambios en la interfaz. Al replicar los mismos coeficientes en leftChain y rightChain, se garantiza una respuesta de fase y magnitud idéntica en ambos canales, manteniendo la coherencia de la imagen estéreo.

7. Integración y Lógica del Filtro LowCut (High-Pass)

Se ha implementado el procesamiento del filtro de corte de graves (LowCut), utilizando una topología de filtro Butterworth de orden variable. La pendiente del filtro (slope) determina cuántos eslabones de la cadena de procesamiento se activan para alcanzar la atenuación deseada.

```
    auto chainSettings = getChainSettings(apvts);
    auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(sampleRate, chainSettings.peakFreq,
        chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));

    auto cutCoefficients = juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod(chainSettings.lowCutFreq,
                                                                                                        getSampleRate(),
                                                                                                        2 * (chainSettings.lowCutSlope + 1));

    *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
    *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;

    // --- CONFIGURACIÓN LOW CUT IZQUIERDO ---
    auto& leftLowCut = leftChain.get<ChainPositions::LowCut>();
    leftLowCut.setBypassed<0>(true);
    leftLowCut.setBypassed<1>(true);
    leftLowCut.setBypassed<2>(true);
    leftLowCut.setBypassed<3>(true);
```

```
switch (chainSettings.lowCutSlope)
{
case Slope::Slope_12:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    break;
}
case Slope::Slope_24:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    break;
}
case Slope::Slope_36:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    *leftLowCut.get<2>().coefficients = *cutCoefficients[2];
    leftLowCut.setBypassed<2>(false);
    break;
}
case Slope::Slope_48:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    *leftLowCut.get<2>().coefficients = *cutCoefficients[2];
    leftLowCut.setBypassed<2>(false);
    *leftLowCut.get<3>().coefficients = *cutCoefficients[3];
    leftLowCut.setBypassed<3>(false);
    break;
}
}
```

```
switch (chainSettings.lowCutSlope)
{
case Slope::Slope_12:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    break;
}
case Slope::Slope_24:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    break;
}
case Slope::Slope_36:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    *rightLowCut.get<2>().coefficients = *cutCoefficients[2];
    rightLowCut.setBypassed<2>(false);
    break;
}
case Slope::Slope_48:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    *rightLowCut.get<2>().coefficients = *cutCoefficients[2];
    rightLowCut.setBypassed<2>(false);
    *rightLowCut.get<3>().coefficients = *cutCoefficients[3];
    rightLowCut.setBypassed<3>(false);
    break;
}
}
```

Parámetros y Lógica:

- Diseño del Filtro Butterworth: Se generan los coeficientes para un filtro paso-alto (High-Pass) basándose en la frecuencia de corte (lowCutFreq) extraída del APVTS.

- Cálculo del Orden del Filtro: La función requiere el orden del filtro matemático. Como cada opción de nuestro menú (Slope_12, Slope_24, etc.) representa un índice (0, 1, 2, 3), la fórmula 2 * (chainSettings.lowCutSlope + 1) traduce este índice en los órdenes correctos (2, 4, 6, 8 correspondientemente).

- Gestión de la Cadena (Bypass): Nuestro CutFilter está compuesto por hasta 4 filtros individuales (cada uno aportando 12 dB/Oct). Por defecto, se ponen todos en estado de bypass (apagados) al inicio del bloque.

Funciones y Variables Usadas:

- *juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod*: Retorna un ReferenceCountedArray que contiene los coeficientes para los distintos filtros IIR en cascada necesarios para formar el filtro de orden superior.

- *setBypassed<Index>(bool)*: Método de juce::dsp::ProcessorChain que activa (false) o desactiva (true) un procesador específico dentro de la cadena sin sacarlo del flujo de la memoria, ahorrando CPU cuando no se necesitan pendientes pronunciadas.

- Desreferenciación de Coeficientes (**cutCoefficients[i]*): Extrae el bloque de coeficientes específico del array generado para asignarlo a la posición i de nuestra subcadena LowCut.

La implementación de pendientes variables se logra mediante la cascada de múltiples filtros IIR de segundo orden (bicuadráticos). Al utilizar la aproximación de Butterworth, garantizamos una respuesta en frecuencia lo más plana posible en la banda de paso (sin rizado) a expensas de una transición de fase no lineal. El apagado dinámico de los nodos no utilizados mediante setBypassed optimiza los ciclos de CPU, asegurando que el motor de audio solo procese las operaciones matemáticas estrictamente necesarias para la pendiente seleccionada (ej. 1 filtro para 12 dB/Oct, 4 filtros para 48 dB/Oct).